Homework #1 (5%)

Submission deadline: Thursday, **February 26, 2026 (23:59)**

Important Notes (must read):

1- You are required to submit your work only through Blackboard. Submissions by email or by any other means will not be accepted**.**
2- This assignment must be completed individually. You must not discuss solutions with other students, write solutions together, or copy and paste any part of the solution that is not your own work.
3- If plagiarism is detected, a mark of zero (0) will be given for the entire assessment.
4- You are required to submit your work before the announced deadline, following the instructions provided. Late submissions will not be accepted**.**
5- You must submit one MS Word document and separate Java source file(s) for your program. Place all files in a single folder named Homework1_QUID, compress the folder, and upload the compressed file to Blackboard.
6- In the MS Word document, you must include screenshots showing the input and output of your program.
7- If you have any questions, you should first contact the Teaching Assistant assigned to your section.

Question: Multi-Way Merge Sort and Experimental Analysis

The purpose of this assignment is to improve your understanding of the **Divide and Conquer** (D&C) paradigm through the design, implementation, and experimental evaluation of multi-way merge sort algorithms. A standard two-way merge sort example has already been discussed and shared during the course. In this assignment, you are required to focus on the implementation of multi-way variants.

You are required to implement both a **three-way merge sort** and **a four-way merge sort** algorithm using the **divide and conquer strategy**. Each algorithm should divide the input array into approximately equal-sized subarrays, recursively sort the subarrays, and then merge the sorted subarrays into a single sorted array. Your implementation must correctly handle **uneven splits, duplicate values, and arbitrary input sizes**. Iterative solutions or the use of built-in sorting methods **are not allowed**. For performance evaluation, you are required to compare the execution time of your three-way and four-way merge sort implementations with a two-way merge sort baseline. The two-way merge sort may be reused from the example provided in the course. However, the three-way and four-way implementations **must be your own work.**

When measuring execution time in Java, the first few executions are often inaccurate because the **Java Virtual Machine (JVM)** optimizes the code while it is running. These optimizations include just-in-time (JIT) compilation and other runtime improvements. As a result, early timing results may not represent the true performance of the code. To address this issue, **warmup runs** are used. **Warmup runs** execute the code several times before any measurements are taken, giving the JVM enough time to apply its optimizations. The execution times from these runs are ignored and are not included in the final results.

A simple code example demonstrating warmup runs is shown below. For this assignment, however, you must choose different values for the *warmups* and *runs* variables. In addition, the example code should be modified as needed to fit your own experiments.

```
int warmups = 10; // choose any value between 5 and 10 (including 5 and 10)

int runs = 30;      // choose any value between 20 and 30 (including 20 and 30)

int[ ] base = generateRandomArray(n); // your own function

// Warmup runs (not timed)

for (int w = 0; w < warmups; w++) {

   int[ ] a = base.clone();

   threeWayMergeSort(a);

}

// Timed runs

long total = 0;

for (int r = 0; r < runs; r++) {

   int[ ] a = base.clone();

   long t0 = System.nanoTime();

   threeWayMergeSort(a);

   long t1 = System.nanoTime();

   total += (t1 - t0);

}

double avgMs = (total / (double) runs) / 1_000_000.0;

System.out.println("Average time (ms): " + avgMs);
```

You are required to conduct the experimental evaluation using **multiple input sizes**. The smallest input size must be at least **50,000**, and the largest input size must be at least **1,600,000**. You must select the **intermediate input sizes yourself**. Each experiment must be repeated multiple times, and the average execution time should be reported. You may present your results using a table or a figure. You are required to submit your Java source code together with a short report. In the report, you must analyze the running time of your three-way and four-way merge sort algorithms and derive their time complexity. You must also present your experimental results and explain what you observe when comparing the two-way, three-way, and four-way approaches. All submissions will be checked for code similarity. Highly similar implementations, including identical experimental settings, may be considered a violation of academic integrity.